

大规模分布式GPU训练



1. 概述

本wiki主要整理两篇分布式异构训练（多GPU和多TPU）的论文《[ImageNet/ResNet-50 Training in 224 Seconds](#)》和《[Image Classification at Supercomputer Scale](#)》。

核心观点是：

- batch size control & distributed batch normalization优化global batch size带来的精度损失
- dataset sharding and caching, prefetch等多种措施优化pre-processing
- 2d-torus all-reduce优化梯度同步

2. batch size control & distributed batch normalization

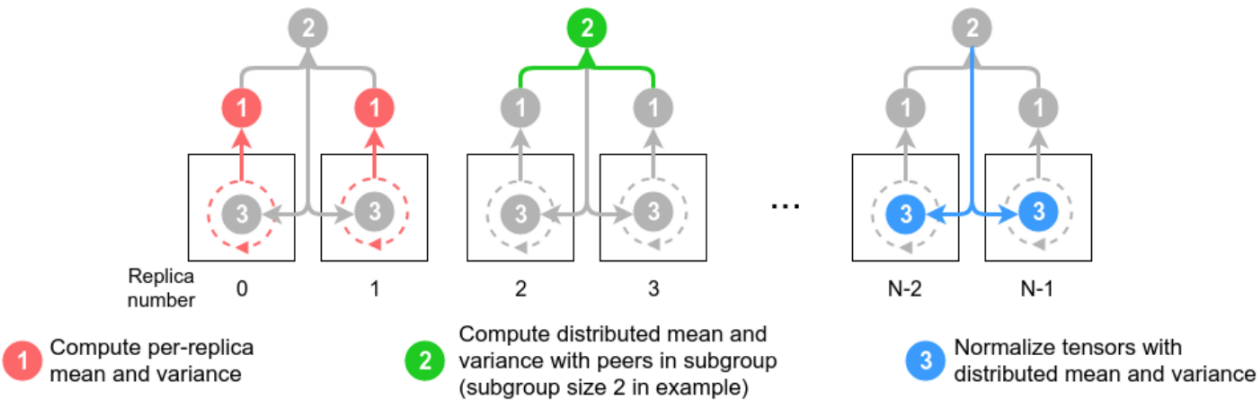
2.1 问题

为了提高训练性能，会增加更多的卡，该行为会导致global batch size的增加。而模型精度与global batch size和per-worker batch size相关。global batch size越大，或per-worker batch size小于32，会影响训练精度。

2.2 方法

batch size control: 随着训练的进行，慢慢的增加global batch size

distributed batch normalization: batch normalization（**BN**就是通过一定的规范化手段，把每层神经网络任意神经元这个输入值的分布强行拉回到均值为0方差为1的标准正态分布，使得非线性变换函数的输入值落入对输入比较敏感的区域，以此避免梯度消失问题）是图像算法很重要的op，论文指出当batch size小于32进行bn时，会影响模型精度。论文提出分布式bn。



3. preprocessing optimization

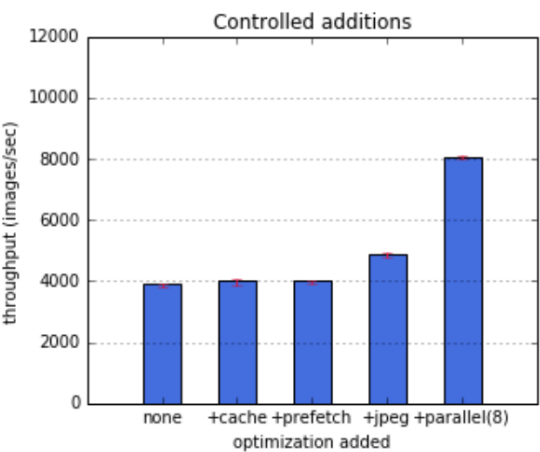
3.1 问题

数据训练之前需要reading, parsing, preprocessing, shuffling, batching等预处理操作，当前异构加速已经很快，预处理操作有可能成为训练的瓶颈。

3.2 方法

- dataset sharding and caching
- prefetch
- fused JPEG decode and cropping
- parallel data parsing

3.3 测试结果



更多资料：[tensorflow dev talk](#)

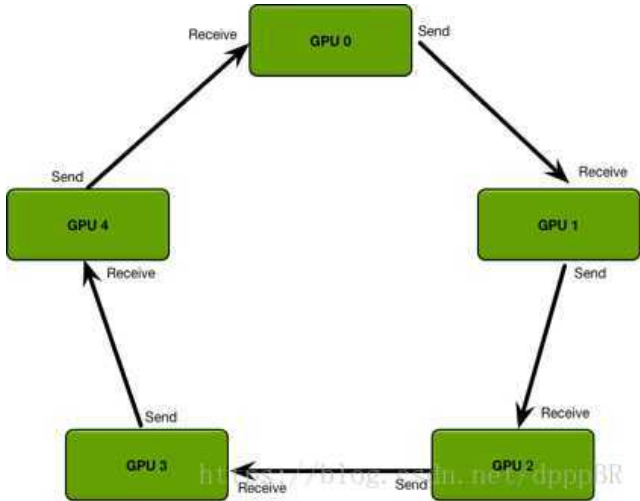
4. 2D-torus all-reduce

4.1 问题

分布式训练中，梯度同步是训练中的一个很大开销。目前梯度同步大部分用all-reduce操作来完成。随着分布式系统节点和异构硬件数的增加，我们需要更加有效的all-reduce操作。

4.2 ring all-reduce

ring all-reduce把所有的节点串成一个环，算法的进行分两步：第一步，scatter-reduce；第二步，allgather。在第一步中，GPU 将交换数据，使得每个 GPU 最终都有一个最终结果的数据块。在第二步中，GPU 将交换那些块，使得所有 GPU 最终得到完整的最后结果。

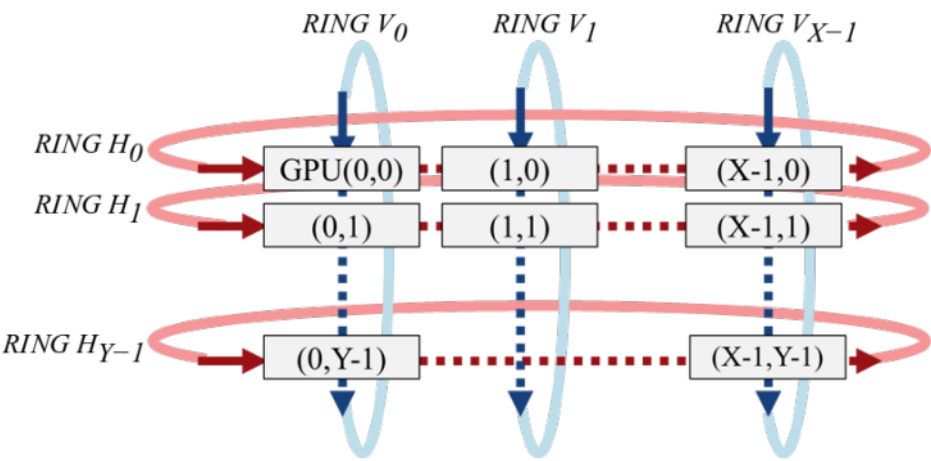


gpu0	a_0	b_0	c_0
gpu1	a_1	b_1	c_1
gpu2	a_2	b_2	c_2
gpu0	a_0	b_0	c_0+c_2
gpu1	a_0+a_1	b_1	c_1
gpu2	a_2	b_1+b_2	c_2
gpu0	a_0	$b_0+b_1+b_2$	c_0+c_2
gpu1	a_0+a_1	b_1	$c_0+c_1+c_2$
gpu2	$a_0+a_1+a_2$	b_1+b_2	c_2

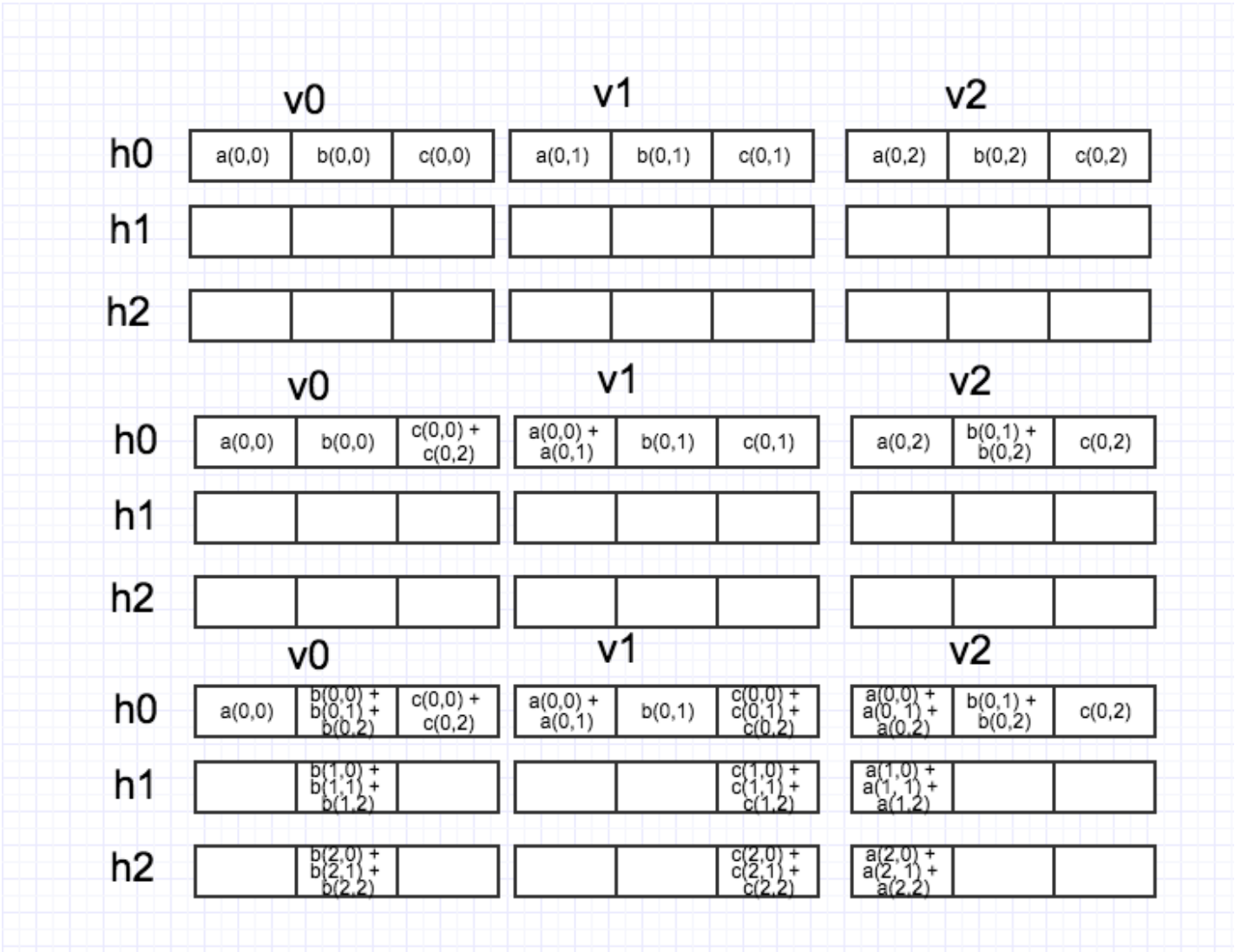
ring all-reduce 是这样一种算法——其通信成本是恒定的，与系统中的 GPU 的数量无关，并且仅由系统中的 GPU 之间的最慢连接来确定。事实上，如果在通信成本上你只考虑带宽这一因素（并忽略延迟），那么 ring allreduce 就是一个最佳的通信算法。

4.3 2d-torus all-reduce

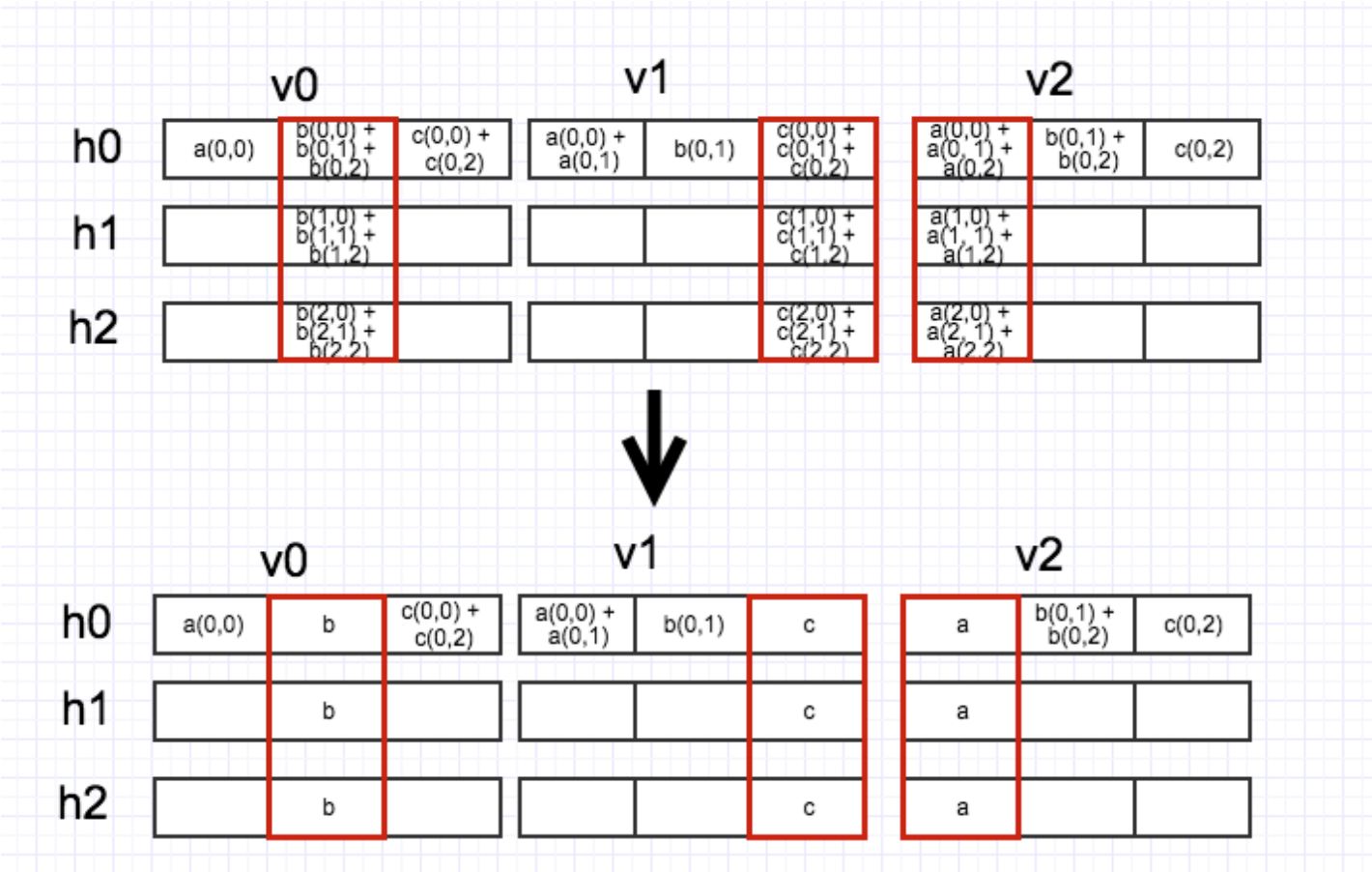
2d-torus all-reduce把节点分成横向的环和纵向的环，该算法的进行分成三步： 第一步， 横向scatter-reduce； 第二步纵向all-reduce; 第三步： 横向all-gather



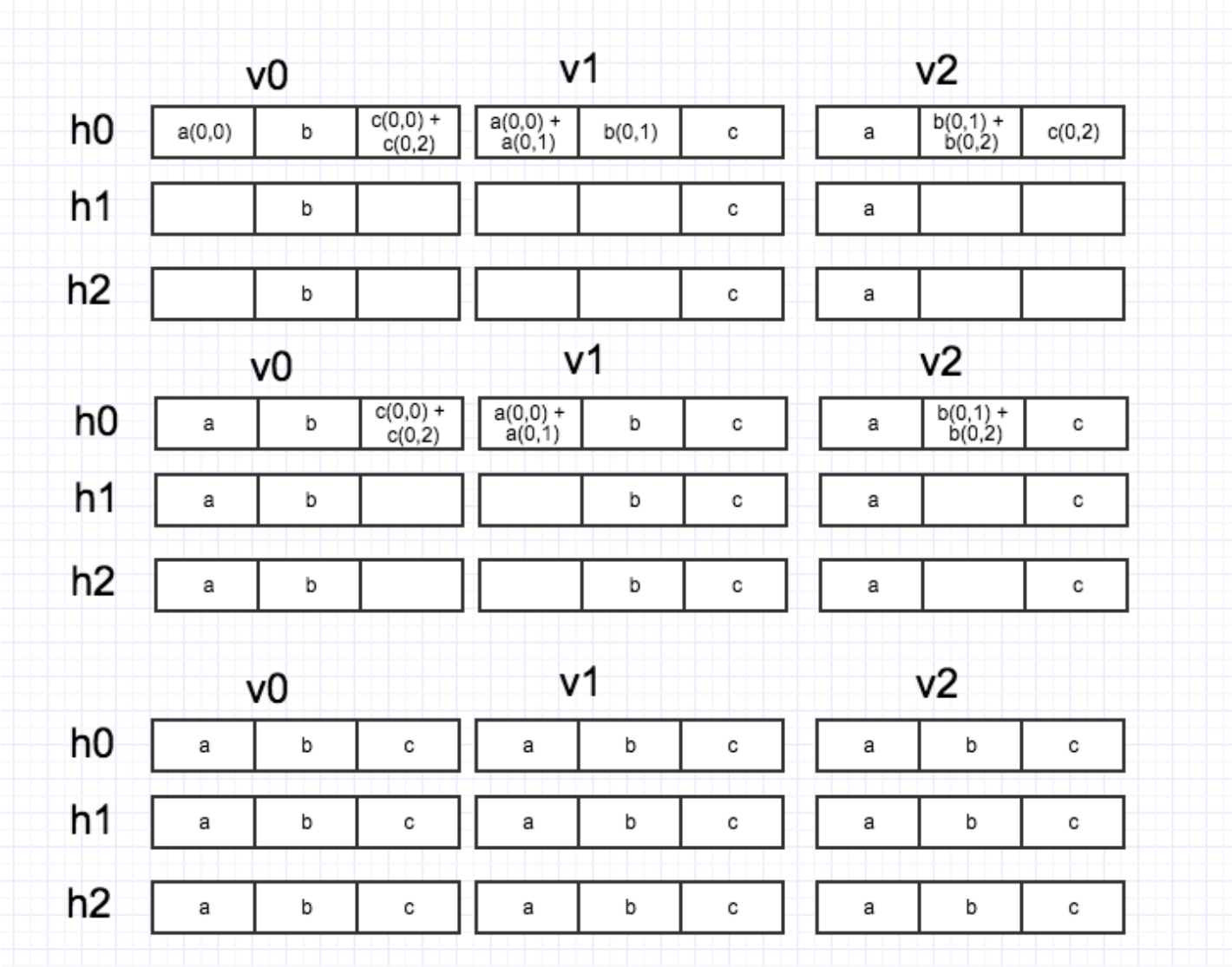
横向scatter-reduce:



纵向all-reduce:



横向all-gather:



其他优化：

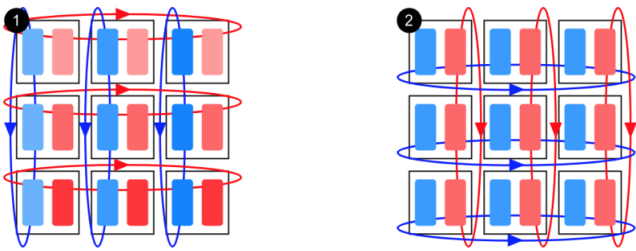


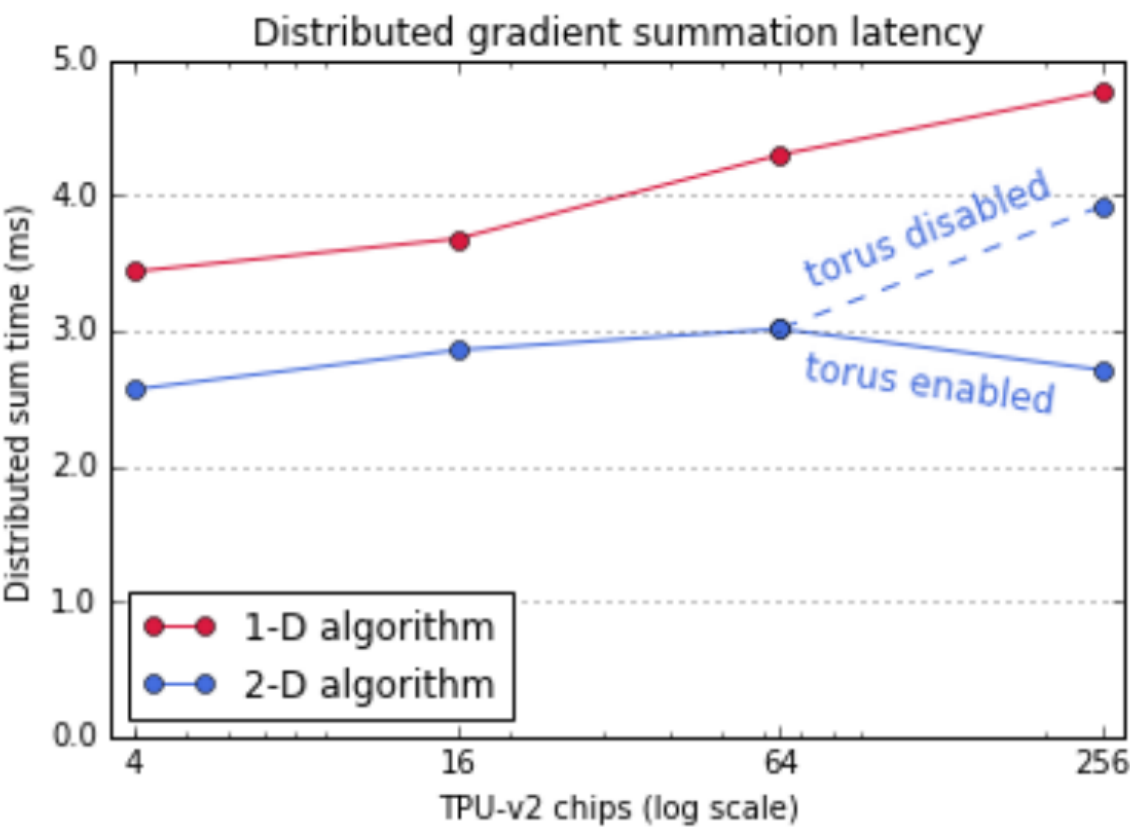
Figure 4: 2-D all-reduce across a hypothetical 3×3 torus. In the first phase (left), the blue tensors are summed along the Y dimension while the red tensors are summed concurrently along the X dimension. In the second phase (right), the dimensions are flipped, which completes the all-reduce for both halves.

4.4 性能分析

假设有 n^2 个节点，每个节点需要同步的数据为 B ，网络带宽为 N 。

1		ring all-reduce	2d-torus all-reduce
2	网络传输时间	$2B(1-1/n^2)/N$	$2B(1-1/n^2)/N$
3	计算时间	$B(1-1/n^2)$	$B(1-1/n^2)$
4	数据传输量(整个网络)	$2B(n^2-1)$	$2B(n^2-1)$
5	数据传输数	$2(n^2 - 1)$	$4(n-1)$

4.5 测试数据



5. 其他测试数据

Table 1 : Training time and top-1 1-crop validation accuracy with ImageNet/ResNet-50

	Batch Size	Processor	DL Library	Time	Accuracy
He et al. [7]	256	Tesla P100 x8	Caffe	29 hours	75.3%
Goyal et al. [1]	8K	Tesla P100 x256	Caffe2	1 hour	76.3%
Smith et al. [4]	8K→16K	full TPU Pod	TensorFlow	30 mins	76.1%
Akiba et al. [5]	32K	Tesla P100 x1024	Chainer	15 mins	74.9%
Jia et al. [6]	64K	Tesla P40 x2048	TensorFlow	6.6 mins	75.8%
This work	34K→68K	Tesla V100 x2176	NNL	224 secs	75.03%

Table 1: Large-scale ResNet-50 training results.

	Hardware	Chips	Batch	Optimizer	BN	Accuracy	Time
Goyal et al. [6]	P100	256	8192	Momentum	Local	76.3%	1 hour
Smith et al. [16]	TPU v2	128	8192 → 16384	Momentum	Local	76.1%	30 mins.
Akiba et al. [2]	P100	1024	32768	RMS + Mom.	Local	74.9%	15 mins.
Jia et al. [10]	P40	1024	65536	LARS	Local	76.2%	8.7 mins.
Baseline	TPU v2	4	1024	Momentum	Local	76.3%	8.0 hours
Ours	TPU v2	256	16384	Momentum	Local	75.1%	10 mins.
Ours	TPU v2	256	32768	LARS	Local	76.3%	8.5 mins.
Ours	TPU v3	512	32768	LARS	Local	76.4%	3.3 mins.
Ours	TPU v3	1024	32768	LARS	Distributed	76.3%	2.2 mins.